

DA5400: Foundations of Machine Learning

Topic 4: Regression – Part 2



Outline

- Gradient Descent Approach to Linear Regression
- Supervised Learning Workflow
- Bias Vs Variance in Regression Models
- Regularisation in Regression Models

Recap: OLS Solution to Linear Regression

- **Model Structure:** $\hat{y} = w_0 + w_1x_1 + \dots + w_Dx_D$
- **Known:** Mapped Input ($\mathbf{X}$) and Output Data ($\mathbf{y}$)
- Sum of squared residuals in predicting y is given by:

$$SSR = (\mathbf{y} - \mathbf{X}^*\mathbf{w})^T (\mathbf{y} - \mathbf{X}^*\mathbf{w})$$

- Least squares estimate of $\mathbf{w}$ are the coefficients that minimize SSR

$$\hat{\mathbf{w}} = \underset{\mathbf{w}}{\operatorname{argmin}} SSR$$

- **Result:** Analytical solution to the least squares estimate of $\mathbf{w}$ is given by:

$$\hat{\mathbf{w}} = (\mathbf{X}^{*T} \mathbf{X}^*)^{-1} \mathbf{X}^{*T} \mathbf{y} - \text{Normal Equation}$$

Limitations of Closed Form Solution

- Requires computing $(X^{*T} X^*)^{-1}$
- Computationally expensive for large datasets
- Requires storing all data in memory
- Cannot be used when data is streaming in batches
- Q: Do we really need to find the matrix inverse?
- Not really. OLS is fundamentally an optimization problem and can be solved iteratively

Gradient Descent for Linear Regression

Loss Function and Gradient Descent

- Loss Function for Simple Linear Regression:

$$J(\mathbf{w}) = \frac{1}{2N} (\mathbf{y} - \mathbf{X}^* \mathbf{w})^T (\mathbf{y} - \mathbf{X}^* \mathbf{w})$$

$$J(\mathbf{w}) = \frac{1}{2N} \sum_{i=1}^N (y_i - \mathbf{x}_i^T \mathbf{w})^2$$

- Properties of Loss Function:
 - Quadratic and Convex
 - Global minimum exists
 - Ideal for gradient based optimization

Gradient Descent Approach

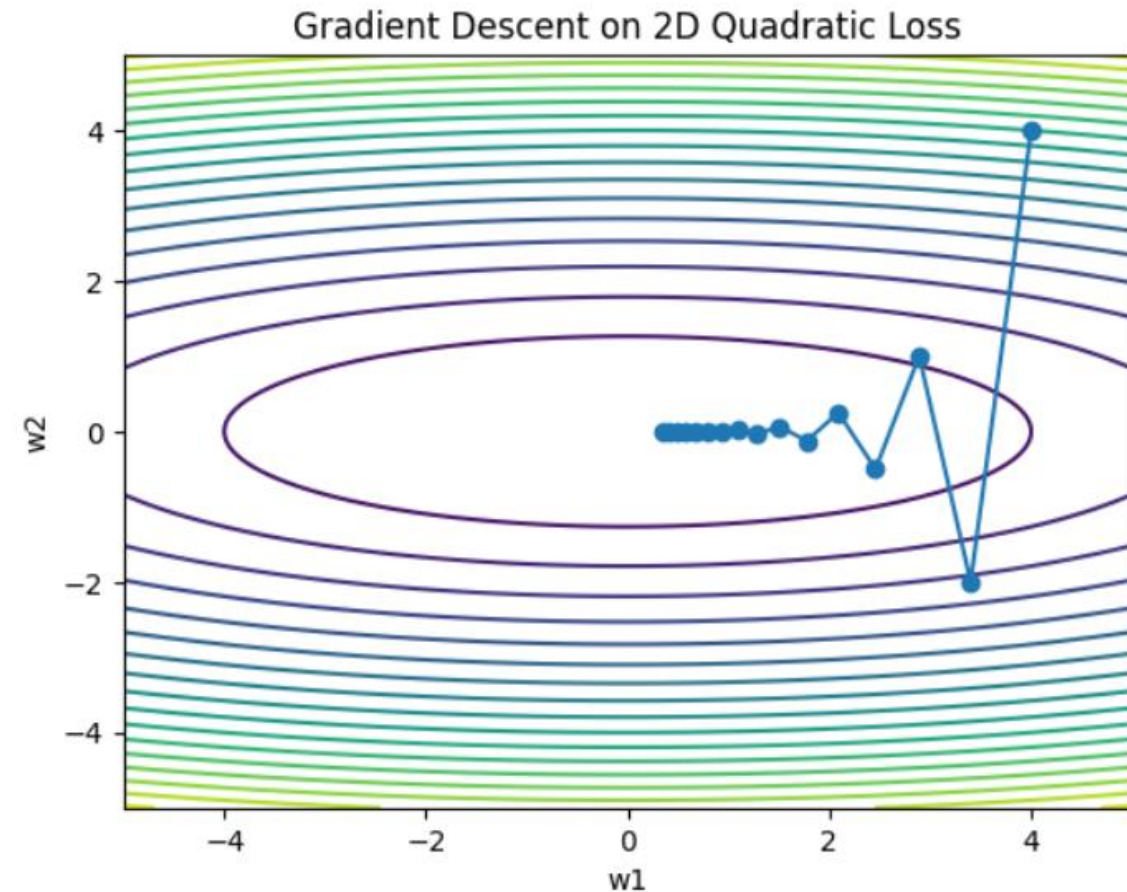
- Principle of Gradient Descent:
 - Start with an initial guess of parameters ($\mathbf{w}$)
 - Iteratively move in the direction of steepest descent of the cost function
$$\mathbf{w}^{k+1} = \mathbf{w}^k - \alpha \nabla J(\mathbf{w}^k)$$

- For Linear Regression:

$$\nabla J(\mathbf{w}) = -\frac{1}{N} \mathbf{X}^T (\mathbf{y} - \mathbf{X}\mathbf{w})$$

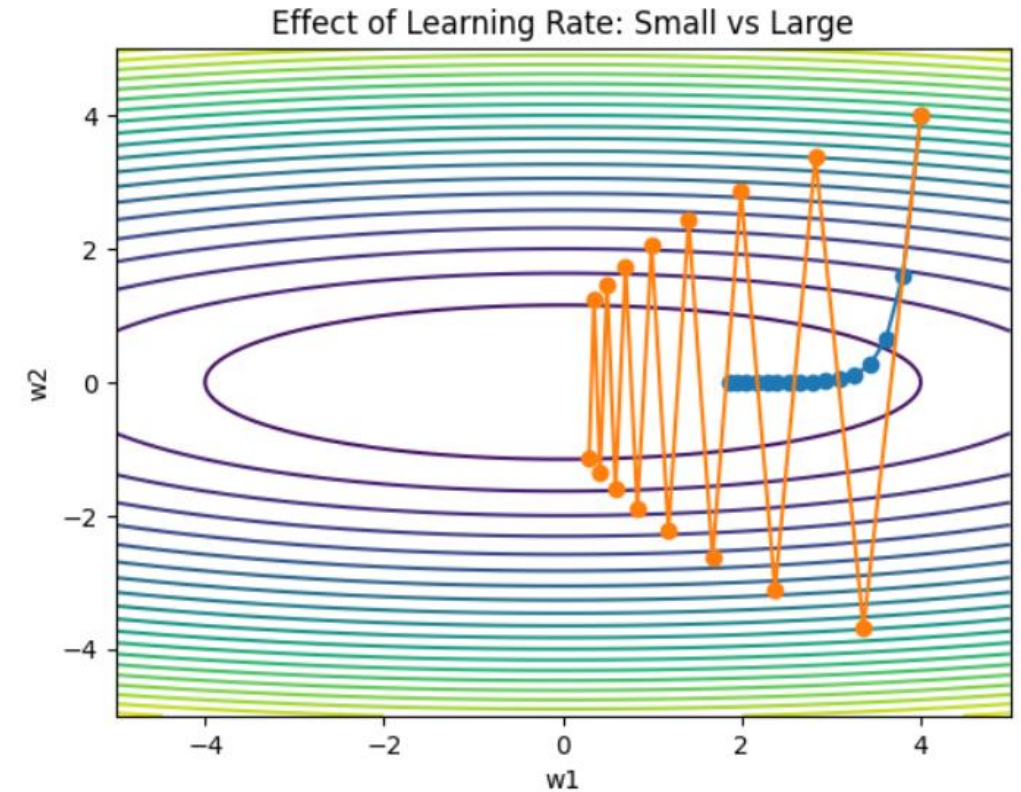
- Gradient update rule:

$$\mathbf{w}^{k+1} = \mathbf{w}^k + \alpha \frac{1}{N} \mathbf{X}^T (\mathbf{y} - \mathbf{X}\mathbf{w})$$



Learning Rate α

- Gradient gives only the direction of steepest descent, not how far to move
- α controls the step size along the gradient
- Choice of alpha:
 - Should be balanced and not too small or too large
 - Too small – Very slow learning
 - Too large – Can jump over the minimum or oscillate and diverge

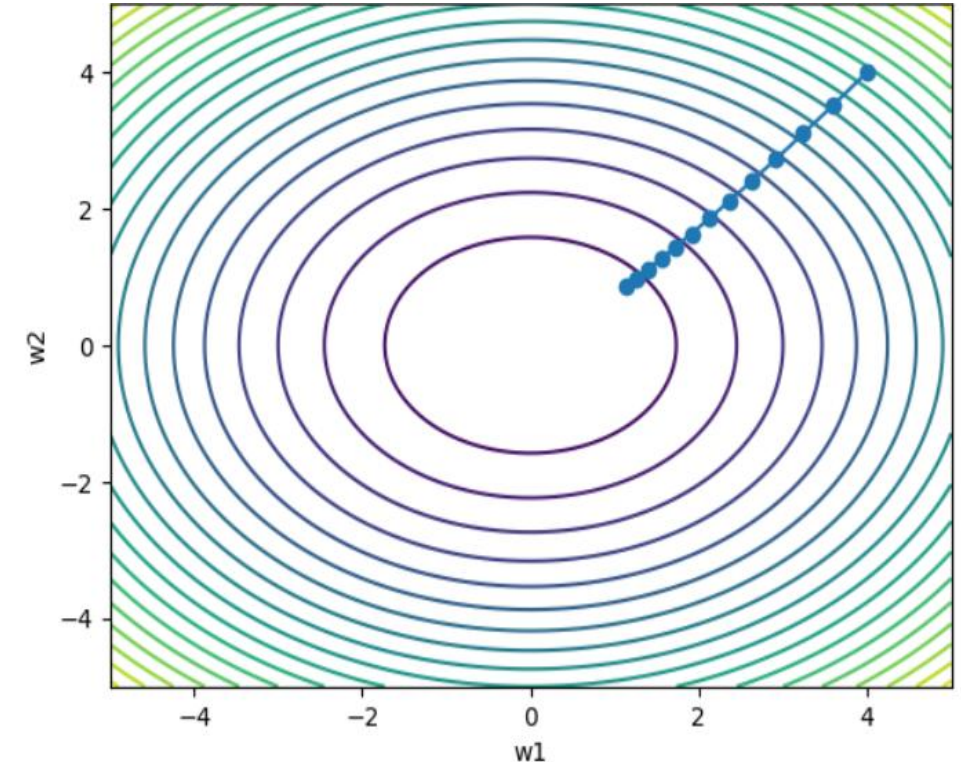


Feature Scaling

- Features are recommended to be scaled to standard normal in gradient descent approach

$$x' = \frac{x - \bar{x}}{\sigma_x}$$

- Without scaling, features with large magnitude dominate the gradient
- Geometrically without scaling:
 - Contours are more elliptical
 - Learning rate should be lower for stability
 - Gradient descent would be slow to converge
- With scaling:
 - Contours are more circular
 - Learning rate can be higher as gradient closely aligns with direction to minimum
 - Smooth and short path to convergence



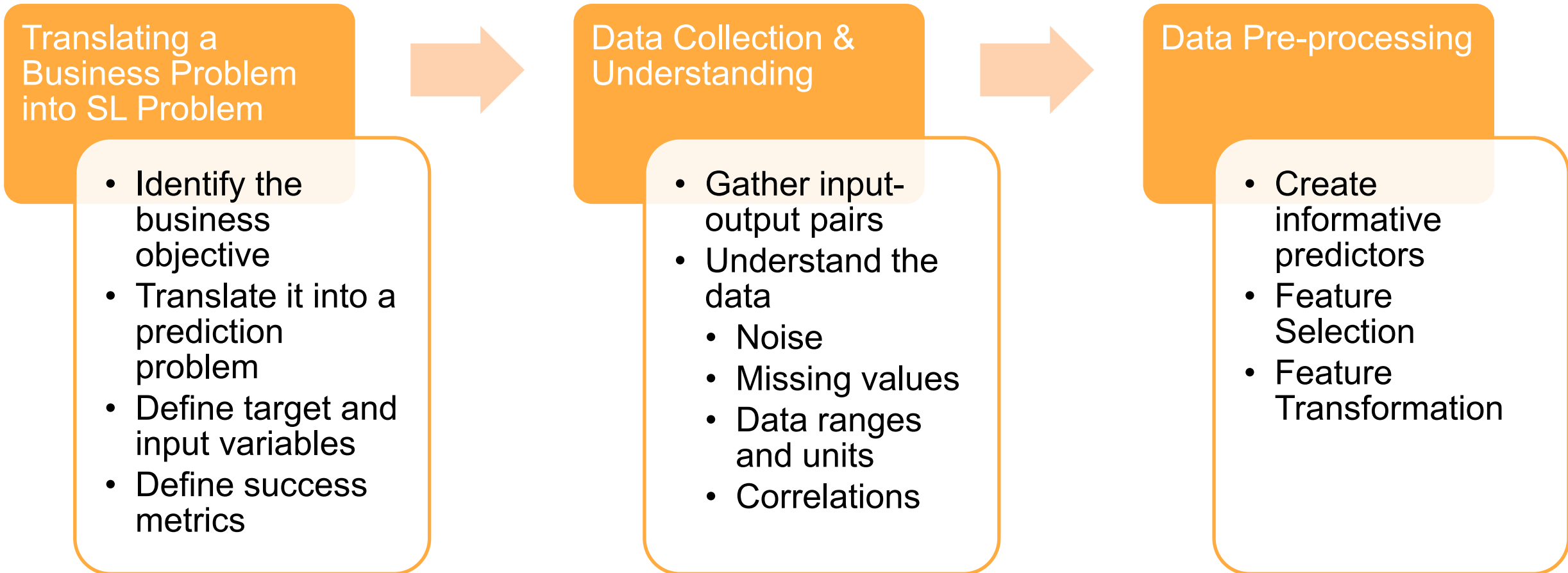
Gradient Descent and Normal Equation

- Gradient descent converges to the same solution as OLS when
 - learning rate is chosen properly
 - enough iterations are run
- Q: How can this be proven mathematically?
- At optimum:

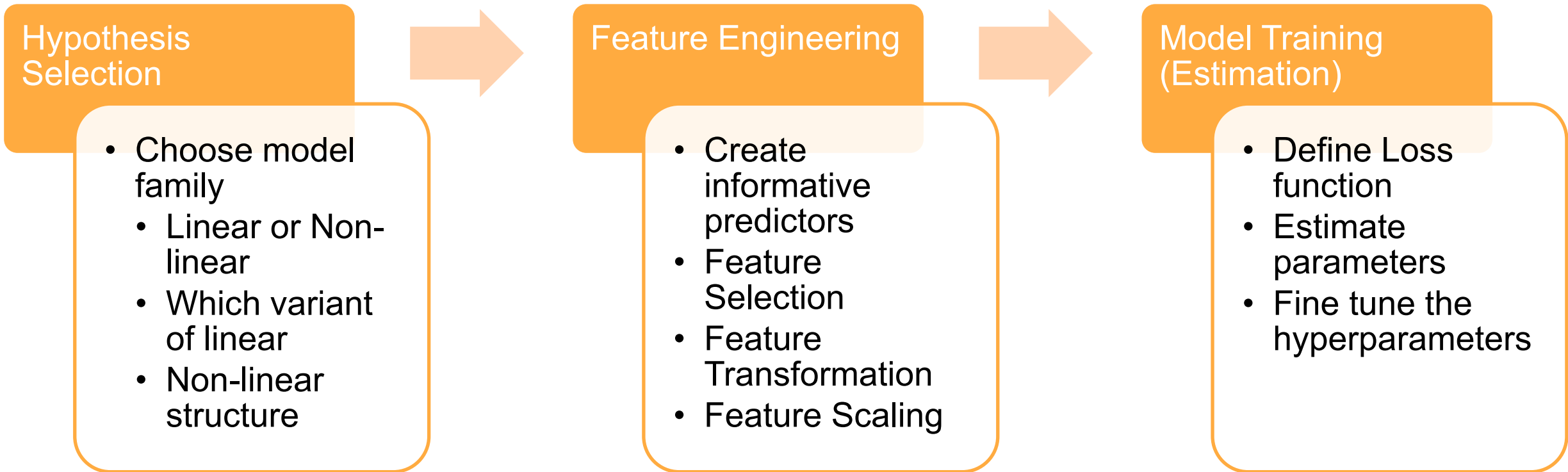
$$\begin{aligned}\nabla J(\mathbf{w}) &= 0 \\ \frac{1}{N} \mathbf{X}^T (\mathbf{y} - \mathbf{X}\mathbf{w}) &= 0 \\ \hat{\mathbf{w}} &= (\mathbf{X}^{*T} \mathbf{X}^*)^{-1} \mathbf{X}^{*T} \mathbf{y}\end{aligned}$$

Supervised Learning Workflow

Supervised Learning Workflow



Supervised Learning Workflow



Supervised Learning Workflow

Model Validation

- Evaluate on data not used for training based on different metrics
- Detect underfitting and overfitting
- Repeat from hypothesis selection if needed

Model Selection & Testing

- Choose model with best validation performance
- Assess performance on a hold out set
- Provides an unbiased estimate of error

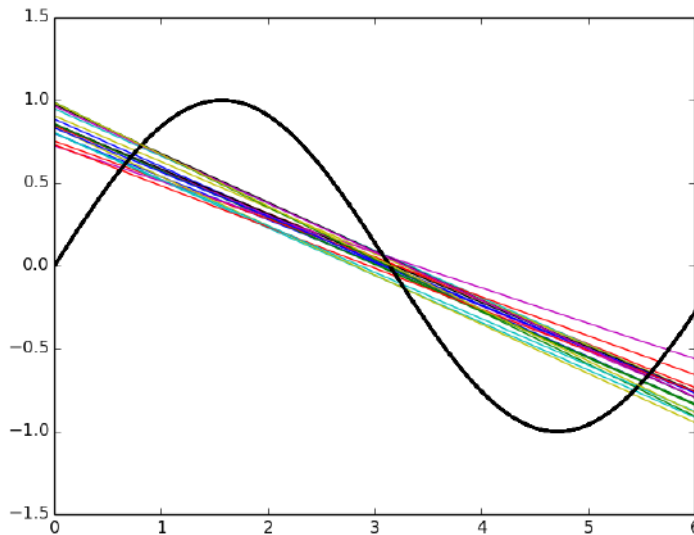
Model Deployment & Maintenance

- Deploy model for predictions
- Monitor the performance regularly
- Check for prediction drift, input distribution drift, need for re-training, etc.

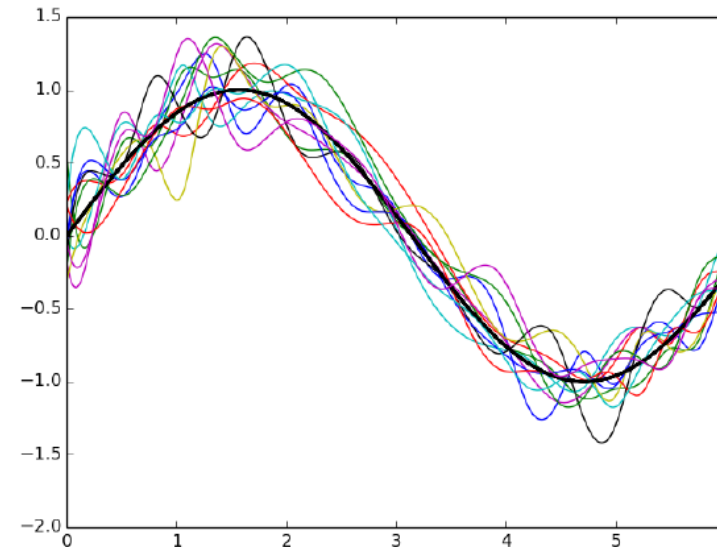
Bias Vs Variance in Regression Models

Bias Vs Variance: Motivation

- Consider a true function $y = \sin x$
- Suppose we draw multiple sample sets (say $N = 30$) with some noise and fit a linear model and high degree polynomial model on each of them



Simple model: Under-fitting

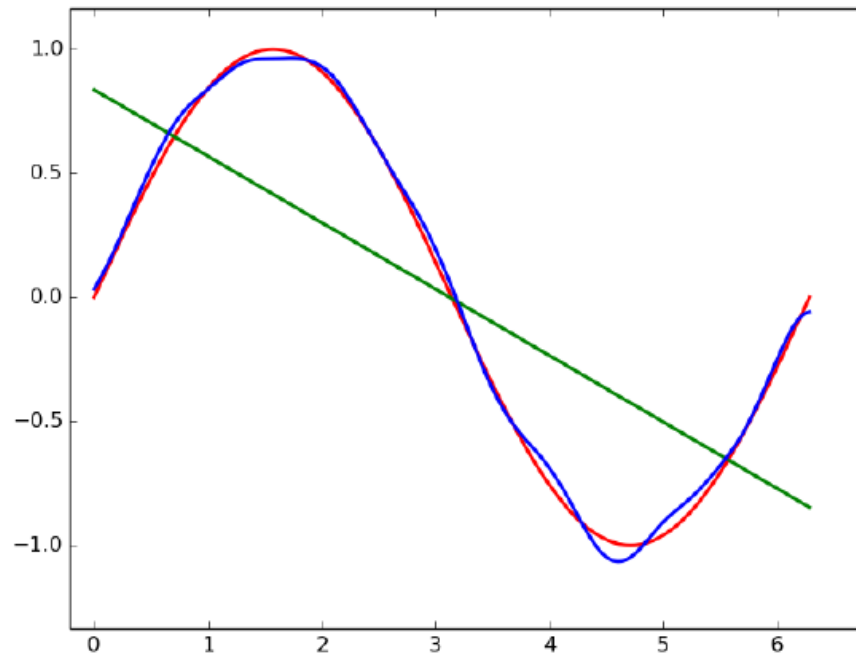


Complex model: Over-fitting

Bias in Predictions

- Bias in predictions measures how far average prediction is from true value

$$\text{Bias} = \mathbb{E}[\hat{y}] - y^t$$



Regression

Green Line: Average value of $\hat{y}$ for a linear model
– High bias

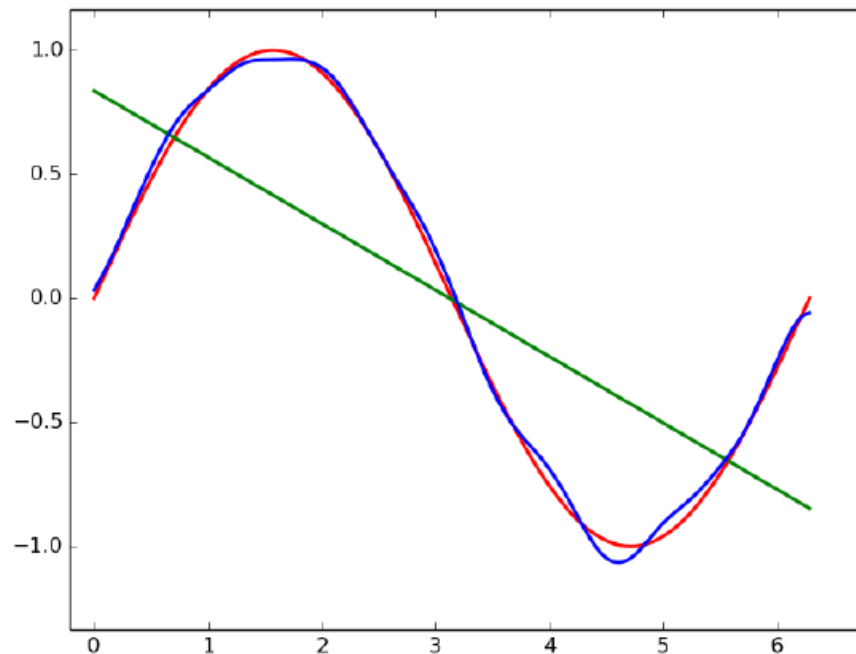
Blue Curve: Average value of $\hat{y}$ for a complex model – Low bias

Red Line: True model (y^t)

Variance in Predictions

- Variance measures change in model's predictions when trained on different datasets

$$Var = \mathbb{E}[\hat{y} - \mathbb{E}[\hat{y}]]^2$$



Regression

Simple model: High bias, Low Variance

Complex model: Low bias, High Variance

So bias Vs variance trade-off is inevitable in ML

Prediction Error due to Bias and Variance

- Prediction bias and variance have two distinct sources
- Model structure (Choice of hypothesis space):
 - Linear Vs Non-linear
 - Polynomial degree
 - Feature set
- Parameter Estimation:
 - Sample variability
 - Noise in observations
 - Multi-collinearity
- **Note:** Increasing data reduces variance but cannot remove model bias

Relation: Error, Bias and Variance

- Consider a new point (x, y) not used in training
- MSE in predicting the value of y using the trained model can be written as:

$$\text{MSE} = \text{Bias}^2 + \text{Variance} + \text{Irreducible error}$$

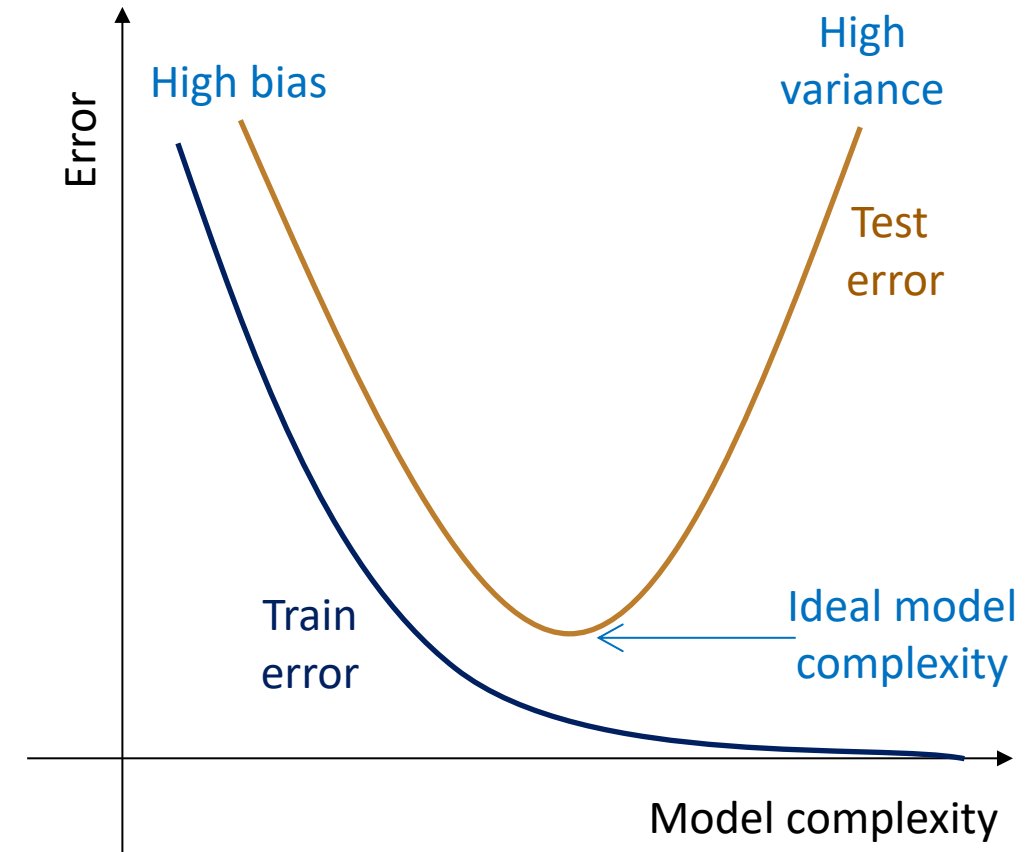
$$\mathbb{E}[(y - \hat{y})^2] = (\mathbb{E}[\hat{y}] - y^t)^2 + \mathbb{E}[(\hat{y} - \mathbb{E}[\hat{y}])^2] + \sigma_\epsilon^2$$

([Check here for proof](#))

Regularised Regression Models

Regularisation: Motivation

- Objectives of Regression:
 - Learn a model that explains training data and generalises to unseen data
 - Bias and variance should be balanced
 - Minimise test error and not just training error
- Idea:
 - Choose a model structure which allows some flexibility (high variance)
 - Regularise it to control that flexibility
 - Converts a high-variance model into a balanced model



Regularisation: Math Idea

- Regularised OLS objective:

$$\min_{\mathbf{w}} (\mathbf{y} - \mathbf{X}^* \mathbf{w})^T (\mathbf{y} - \mathbf{X}^* \mathbf{w}) + \lambda \Omega(\mathbf{w})$$

- $\Omega(\mathbf{w})$: Regularisation penalty – balances against training loss
- $\Omega(\mathbf{w})$ is large for complex models and small for simple models
- $\lambda \geq 0$: Regularisation rate – controls the strength of regularization
- Impact of regularization:
 - Parameters are pushed towards zero
 - Significant decrease in variance with small increase in bias

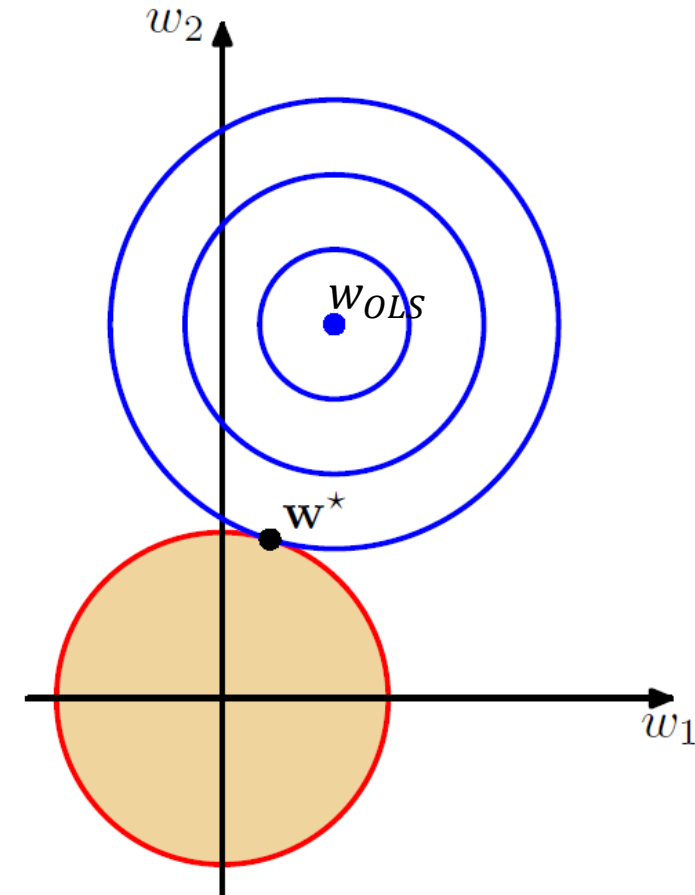
Ridge Regression (L2 Regularisation)

- Ridge regression objective:

$$\min_{\mathbf{w}} (\mathbf{y} - \mathbf{X}^* \mathbf{w})^T (\mathbf{y} - \mathbf{X}^* \mathbf{w}) + \lambda \|\mathbf{w}\|_2^2$$

$$\mathbf{w}_{ridge} = (\mathbf{X}^{*T} \mathbf{X}^* + \lambda \mathbf{I})^{-1} \mathbf{X}^{*T} \mathbf{y}$$

- Penalises squared magnitude of coefficients
- Shrinks all coefficients smoothly with increase in λ
- Shrinks the larger coefficients more
- Geometric View:
 - OLS: Minimises error in an unconstrained space
 - Ridge: Minimises error constrained inside a L2 circle



Diameter of circular contour depends on λ

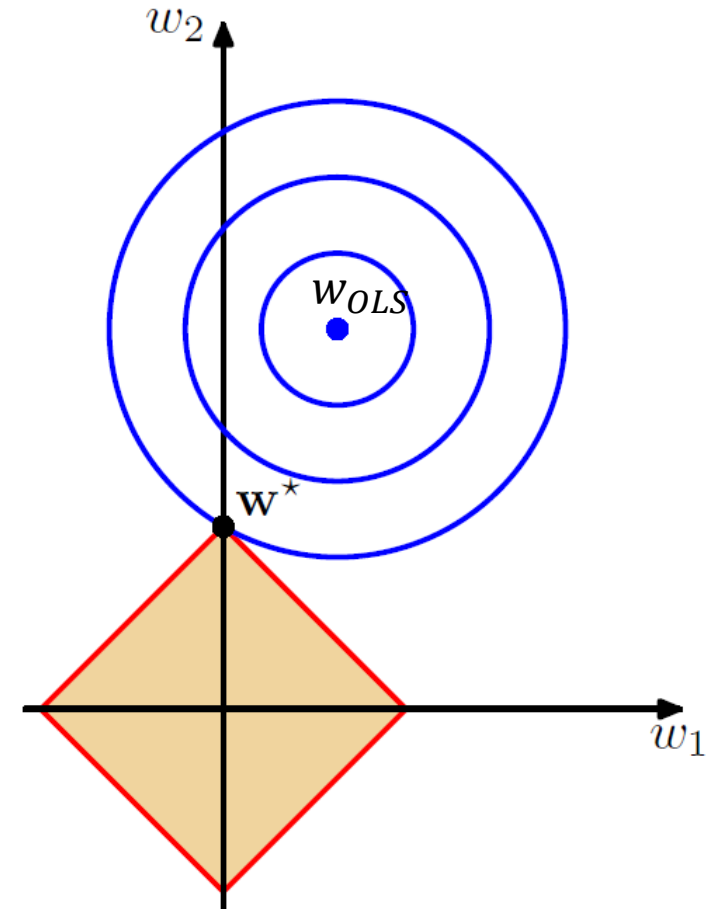
LASSO (L1 Regularisation)

- Least Absolute Shrinkage and Selection Operator

- **LASSO regression objective:**

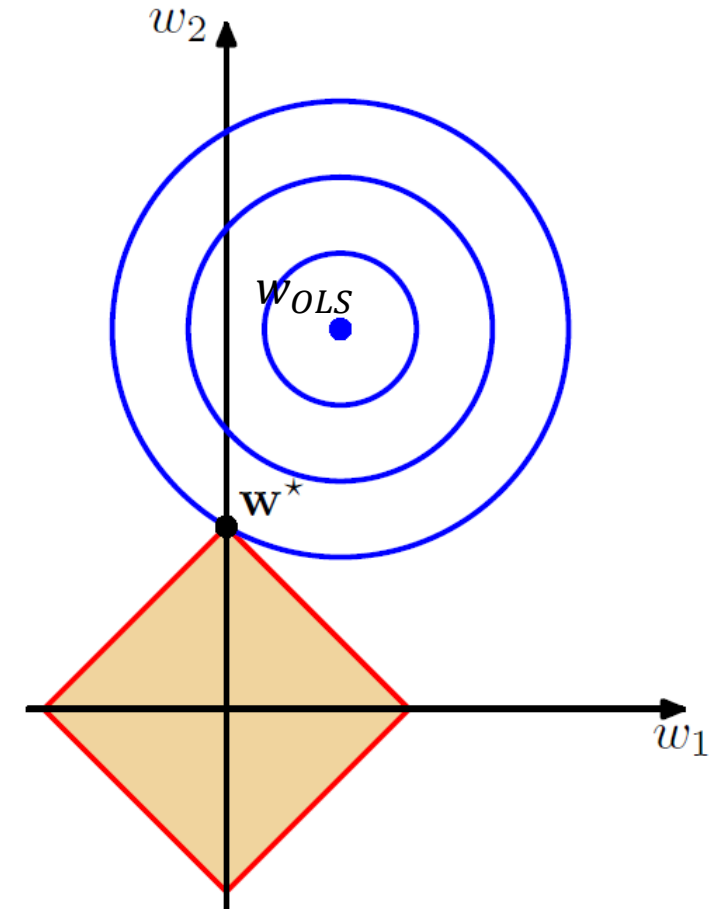
$$\min_{\mathbf{w}} (\mathbf{y} - \mathbf{X}^* \mathbf{w})^T (\mathbf{y} - \mathbf{X}^* \mathbf{w}) + \lambda ||\mathbf{w}||_1$$

- Can drive some coefficients exactly to zero
- Can be used for feature selection
- Geometric View:
 - OLS: Minimises error in an unconstrained space
 - Ridge: Minimises error constrained inside a L1 square



LASSO Solution using Coordinate Descent

- **Issue:** LASSO objective is not differentiable at zero and has no closed form solution
- **Idea of Coordinate Descent:**
 - Optimize the objective one parameter at a time while holding the others fixed
- **Algorithm Steps:**
 1. Standardise each input feature
 2. Initialise with random small values (or zeros) of parameters w^0



LASSO using Coordinate Descent

- Algorithm Steps:

- 3. Repeat until convergence:

- a) For each of the D predictors (inputs), compute partial residual for all samples

$$r_{i(-j)} = y_i - \sum_{k \neq j} x_{ik} w_k \quad \forall j \text{ vars}$$

- Gives what part of y is left unexplained if we ignore variable j
 - This is the target w_j must explain

- b) Compute OLS like contribution of feature j

- Regress this residual only on x_j (all samples)

$$\begin{aligned} \mathbf{r}_{-j} &= \rho_j \mathbf{x}_j \\ \rho_j &= \frac{\mathbf{x}_j^T \mathbf{r}_{-j}}{\mathbf{x}_j^T \mathbf{x}_j} = \mathbf{x}_j^T \mathbf{r}_{-j} \text{ (scaled features)} \end{aligned}$$

- Explains potential value for w_j when there is no L1 penalty

LASSO using Coordinate Descent

- Algorithm Steps:

- 3. Repeat until convergence:

- c) Apply Soft Thresholding and update the weights as follows:

$$w_j = \frac{\rho_j - \lambda}{\|x_j\|^2} ; \rho_j > \lambda$$

$$w_j = 0 ; |\rho_j| \leq \lambda$$

$$w_j = \frac{\rho_j + \lambda}{\|x_j\|^2} ; \rho_j < -\lambda$$

- No gradients, no matrix inverse and natural sparsity
 - Note: λ is chosen using validation techniques

Model Validation

Model Validation

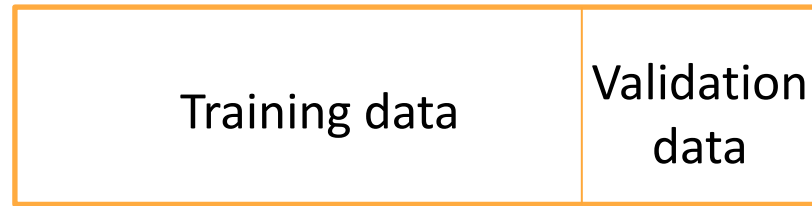
- Hyperparameters are to be chosen by the user
 - Regularisation rate
 - Regularisation type: Ridge or LASSO
 - Learning rate
 - Polynomial degree
- Alternately, it is chosen using a model validation strategy
- To perform model validation and testing, the given dataset is split into three subsets:
 - **Training set**: Set of samples used to learn the model parameters. So loss function is minimised over this dataset
 - **Validation set**: Set of samples used for model comparison and selection
 - **Test set**: Set of samples used to compute unbiased performance of the selected model

Model Validation Techniques

- Different model validation techniques exist which can be used to choose a good model:
 - Hold out
 - K-fold cross validation
 - Leave-one-out cross validation
- All validation techniques involve splitting the given data into training and validation sets and validating using one of the metrics

Model Validation: Hold out

Given Data – N Samples



- Most basic validation method where given data is split into training and validation sets in the ratio of 80:20 or 70:30
- **Issue:** Randomly splitting the data may result in sampling bias
- The distribution of training and validation data sets may not represent the same population distribution from the original data is drawn

Model Validation: K-fold and Leave-one-out

Given Data – N samples

Fold 1	Fold 2			Fold k
--------	--------	--	--	--------

- If there are N samples in the dataset, then they are split into k folds each with $\frac{N}{k}$ samples
- Each time, one fold is kept out of the training set and is used to validate the model after training
- Repeated by considering each of the folds as validation sets and average performance is taken
- This reduces sampling bias to considerable extent

Given Data – N Samples

Fold 1	Fold 2			Fold N
--------	--------	--	--	--------

- A special case of k fold cross validation in which $k = N$
- Used when number of samples are very less in number

Metrics for Regression Model Evaluation

Metrics to Measure Prediction Error

- **Mean Absolute Error (MAE):** Mean of the absolute values of residuals

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- Easy to understand and same units as dependent variable
- Treats all errors equally and does not amplify large errors or outliers
- **Root Mean Squared Error (RMSE):** Square root of the mean of squared residuals

$$RMSE = \sqrt{\left(\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \right)}$$

- Penalizes large errors and useful to detect outliers or when large errors are critical
- Same unit as output

Metrics to Measure Relative Error

- **Mean Absolute Percentage Error (MAPE):** Error as percentage of actual value of dependent variable

$$MAPE = \frac{1}{N} \sum_{i=1}^N \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100$$

- Useful for understanding relative errors and easy to interpret since it is percentage
- Can be very high when actual values are small - to be avoided
- Undefined when $y_i = 0$
- Scale independent and can be used to compare models and outputs at different scales

Metrics to Evaluate Goodness of Fit

- **R-squared (R^2):** Metric which indicates how good is the fit

$$R^2 = 1 - \frac{RSS}{TSS} = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2}$$

- TSS: Represents total variation in the dependent variable relative to its mean
- RSS: Represents the unexplained variation in the dependent variable by the model
- Indicates proportion of variance in the output that is explained by the model
- R^2 of 1 indicates perfect fit where there are no residuals
- **Negative R^2** indicates worse fit where mean prediction is better than model prediction
- **Note:** To be used only to understand variance explained and compare models on same dataset
- **Issues:**
 - Can be misleading in case of low variance in the dependent variable
 - R^2 increases with additional variables even if irrelevant
 - High R^2 does **not imply good predictions** (model may still have large errors)

Feature Selection Methods

Feature Selection

- **Objective:** Select a subset of the features that minimises generalisation or validation error

$$S \subset \{1, 2, \dots, D\}$$
$$\mathcal{L}_{val}(S) = \frac{1}{N_{val}} \sum_i \left(y_i - \hat{y}_i^{(s)} \right)^2$$

- **Issue:** Search over all possible subsets is not feasible
- Approximate approaches:
 - **Recursive Feature Elimination:** Start with all features and remove least important steps in a recursive fashion
 - **Sequential Feature Selection:** Build subset step by step based on validation loss
- Computationally expensive but generally better than LASSO

Recursive Feature Elimination

- **Idea:** Important of feature j is proportional to the associated coefficient w_j
- Small $|w_j| \rightarrow$ Weak contribution $\rightarrow$ Candidate for removal
- **Algorithm:**
 - Let $S_0 = \{1, 2, \dots, D\}$
 - For each iteration $i = 0, 1, 2, \dots$
 - Fit Linear model using all features in S_i
 - Rank the features based on their coefficients: $r_j = |w_j^{(i)}|$
 - Remove feature with smallest rank
 - Repeat until desired number of features remain
- Greedy backward elimination as an approximation to best subset selection
- Works well when features are not highly correlated

Sequential Feature Selection

- Here, the validation loss is directly minimized

$$\mathcal{L}_{val}(S) = \frac{1}{N_{val}} \sum_i \left(y_i - \hat{y}_i^{(s)} \right)^2$$

- **Algorithm:**
 - Start with an empty feature set $S_0 = \Phi$
 - At step i :
 - For each candidate feature j not in S_i
 - Fit model using $S_i \cup \{j\}$
 - Compute Validation loss $\mathcal{L}_{val}(S_i \cup \{j\})$
 - Choose feature: $j^* = \underset{j}{\operatorname{argmin}} \mathcal{L}_{val}(S_i \cup \{j\})$
 - Update the feature set: $S_{i+1} = S_i \cup \{j^*\}$
- **Note:** This can also be done in a backward manner by starting with complete set of features
- Use when validation performance is the primary goal and when features are correlated

Bayesian Linear Regression

Bayesian Linear Regression: Key Idea

- Linear Model

$$y = w_0 + w_1\phi_1(\mathbf{x}) + w_2\phi_2(\mathbf{x}) + \cdots + w_D\phi_D(\mathbf{x}) + \epsilon$$

- Assumption on noise: $\epsilon \sim \mathcal{N}(0, \beta^{-1})$
- Implies target values follow a Gaussian distribution

$$p(\mathbf{y}|\mathbf{w}) = \mathcal{N}(\mathbf{X}\mathbf{w}, \beta^{-1}\mathbf{I})$$

- β : precision or inverse variance
- Bayesian Idea:
 - Estimate a distribution over parameters instead of point estimates
 - Assumes a prior distribution on parameters and infers a posterior based on data using Baye's rule

$$p(\mathbf{w}|\mathcal{D}) = \frac{p(\mathcal{D}|\mathbf{w})p(\mathbf{w})}{p(\mathcal{D})}$$

Prior and Posterior

- Prior Assumption:

$$p(\mathbf{w}) = \mathcal{N}(0, \alpha^{-1} \mathbf{I})$$

- Other Assumptions:

- Inputs are treated as fixed and certain
 - Only the outputs are modelled probabilistically
 - For simplicity $\mathcal{D}$ is replaced with $\mathbf{y}$ in the probabilities
-
- Since prior and likelihood are gaussian, posterior is also a gaussian
$$p(\mathbf{w}|\mathbf{y}) = \mathcal{N}(\mathbf{m}_N, \mathbf{S}_N)$$
 - Instead of a single weight vector, we have a distribution over weights

Posterior Mean and Covariance

- Posterior distribution:

$$p(\mathbf{w}|\mathbf{y}) = \mathcal{N}(\mathbf{m}_N, \mathbf{S}_N)$$

- Posterior Gaussian's Covariance matrix:

$$\mathbf{S}_N = (\alpha \mathbf{I} + \beta \mathbf{X}^T \mathbf{X})^{-1}$$

- **Interpretation:** Posterior precision = Prior precision + Data precision

- Posterior Gaussian's mean vector:

$$\mathbf{m}_N = \beta \mathbf{S}_N \mathbf{X}^T \mathbf{y}$$

$$\mathbf{m}_N = (\alpha \mathbf{I} + \beta \mathbf{X}^T \mathbf{X})^{-1} \beta \mathbf{X}^T \mathbf{y}$$

- **Interpretation:** Equivalent to ridge regression solution with $\lambda = \frac{\alpha}{\beta}$
- In other words, large prior precision or large noise variance acts as strong regularisation
- **Note:** Ridge regression can be interpreted as MAP estimation

Predictions

- Consider a new sample $(\mathbf{x}_{new}, y_{new})$
- From model assumption:
$$p(y_{new}|\mathbf{w}) = \mathcal{N}(\mathbf{x}_{new}^T \mathbf{w}, \beta^{-1})$$
- Above describes the prediction for a fixed $\mathbf{w}$
- Since $\mathbf{w}$ is uncertain with known distribution, prediction is averaged over the posterior

$$p(y_{new}|\mathbf{x}_{new}, \mathcal{D}) = \int p(y_{new}|\mathbf{w})p(\mathbf{w}|\mathcal{D})d\mathbf{w}$$

$$p(y_{new}|\mathbf{x}_{new}, \mathcal{D}) = \mathcal{N}(\mathbf{x}_{new}^T \mathbf{m}_N, \beta^{-1} + \mathbf{x}_{new}^T \mathbf{S}_N \mathbf{x}_{new})$$

Non-Linear Regression

Non-Linear Regression

- Suppose there exists a non-linear regression model which structure is as follows:

$$\ln y = A - \frac{B}{x+C} : \text{Antoine Eq.}$$

- A,B,C are the parameters and x, y are the variables
- Objective is to estimate A, B, C given the data (n samples) pertaining to x, y
- Cannot be converted to a linear form in terms of the parameters

Non-Linear Least Squares Solution

- Objective: Minimise the sum of residuals between observed and predicted values

$$\min_{A,B,C} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$
$$s.t. \ln \hat{y}_i = A - \frac{B}{x_i + C}$$

- Steps:
 - Choose initial values for A, B, C
 - Use a non-linear least squares solver (Gradient descent or Gauss-Newton) to minimise the objective and obtain A, B, C
- Note: scipy library in python has this solver inbuilt

Topics for Further Reading in Regression

- Generalised Least Squares
- Elastic Net : L1 + L2 regularisation of a linear regression model
- Confidence intervals for parameters and predictions